29/08/24

Algorithms (P.B)

Dynamic Programming

Overlapping Subproblem

Optimal Substructure - the small subproblem must be optimally solved.

Matrix chain order Multiplication

Input: $A_1, A_2, A_3, \ldots, A_n$ are matrices

Output: Order of multiplying the matrix $A_1 A_2 \ldots A_n$ such that the # of multiplication is minimum.

* $A_{p \times q} \cdot B_{q \times r}$ = No. of computation pqr

$A_{p \times q} B_{q \times r} C_{r \times s} = pqr + prs$

~~$A_1 A$~~ A	B	C
10x100	100x5	5x50

$(AB) \cdot C = 7500$
$A(BC) = 75000$

Give the optimal parenthisization of matrices A_1 to A_n s.t minimum # of multiplication

$$P(n) = \sum_{K=1}^{n} P(K) \cdot P(n-k)$$

$P(n)$ is no. of parenthesis for n matrices

$$P(n) = 2 \sum_{i=1}^{n-1} P(i) P(n-i) = C_n \text{ (nth cateler no.)}$$

$$= \frac{1}{n+1} \binom{2n}{n} = \Omega\left(4^n / n^{3/2}\right)$$

which is exponential time complexity

That's why we use dynamic programming approach to ~~from~~ find it in $O(n^3)$ complexity.

Dynamic Programming

1) Characterise the problem

2) Recurrence relation formulation

3) computation of optimal cost using bottom up approach.

4) obtaining the optimal problem.

Matrix chain

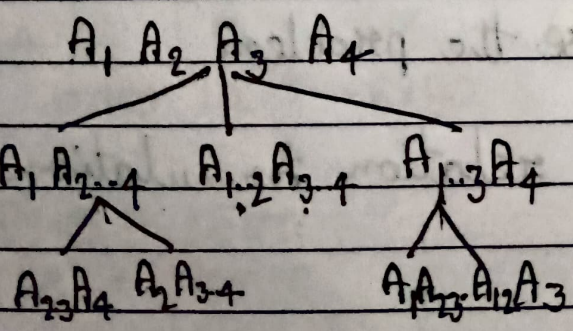
1) structure of optimal parenthisization

$A_{i \ldots j} = A_i A_{i+1} \cdots A_j$

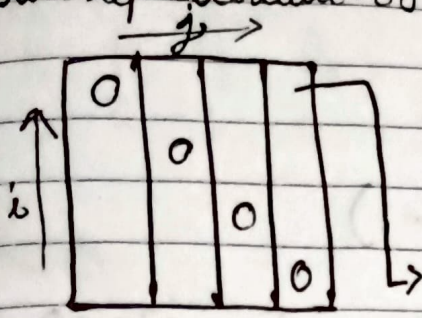
- A n optimal parenthisization splits the product between A_k and A_{k+1} for some integer k, $1 \leq k \leq n$

- Cost of optimal parenthisization

= cost of $A_{1 \ldots k}$ + cost of $A_{k+1 \ldots j}$ + cost of computing $A_{1 \ldots k} \cdot A_{k+1 \ldots j}$



matrix representation of dynamic programming



$m[1,2] = A_{1\cdots 2}$

$m[1,3] = A_{1\cdots 3}$ (best value of $A_1 A_2 A_3$ and $A_1 A_2$)

→ final min no of multiplication

$A_1 \cdot A_{2\cdots 3} = m[1,1] + m[2,3] + P_0 P_1 P_3$

$\therefore A_{i\cdots k} \cdot A_{k+1\cdots j} = \min_{i \le k \le j} \{ m[i,k] + m[k+1,j] + P_{i-1} P_k P_j \}$

(3) matrix chain ~~multiplication~~ $(P[0\cdots n])$

dimension of $A_1 \cdots A_n$

$S[1\cdots n-1, 1\cdots n-1]$

$m[1\cdots n, 1\cdots n]$

SUNDAY 14

for $i=1$ to n

$m[i,i] = 0$

for $L = 2$ to n:

for $(i = 1$ to $n-L+1)$

```
    j = i + L - 1
    m = [i, j] = ∞
    for (k = 1 to j - 1)
        cost = m[i, k] + m[k+1, j] + P[i-1] · P[k] · P[j]
        if cost < m[i, j] {
            m[i, j] = cost
            S[i, j] = K
        }
    }
}
}
return m[1, n] & S[n, n]
```

$(A_1)(A_2, A_3)(A_4)$

P_0	P_1	P_2	P_3	P_4
5	4	6	2	7

	1	2	3	4
	0	120	88	158
		0	48	104
			0	84
				0

S

0		1	3
	0		3
		0	
			0

~~A1..2~~

$A_{1..2} = P_0 \times P_1 \times P_2 = 5 \times 4 \times 6 = 120$

$A_{1..3} = A_{1..2} A_3 = (m[1,1] + m[2,3] + P_0 P_1 P_3 \Rightarrow 88$ ✓

min of

$A_1 A_{2..3} = (m[1,2] + m[3,3] + P_0 P_2 P_3 \Rightarrow 18$

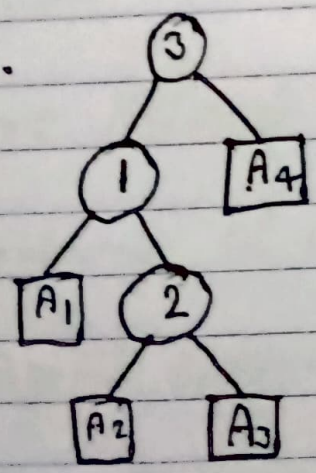
like wise $P_{2..4}$ min(252, 104)

$A_{1..4} = m[1,1] + m[2,4] + P_0 P_1 P_4 = 244$

min of $m[1,2] + m[3,4] + P_0 P_2 P_4 = 414$

$m[1,3] + m[4,4] + P_0 P_3 P_4 = 158$ ✓

$\therefore$



4) Obtain the optimal order

```
do-multi(i,j) {call code (1,4)
    if i=j
        return A[i]
    else {
        K = S[i,j]
        X = do multi(i,K)
        Y = do multi(K+1,j)
        return X*Y
```